

IPD434

Aprendizaje por refuerzo

Deep Reinforcement Learning

Dr. Patricio Olivares Roncagliolo

Ingeniería Civil Telemática · Universidad Técnica Federico Santa María

¿Qué problema resuelve el aprendizaje por refuerzo?

- Un agente aprende a **tomar decisiones secuenciales** al interactuar con un entorno.
- Cada acción cambia el estado y puede producir una recompensa inmediata.
- El objetivo es aprender una estrategia que maximice la **recompensa acumulada**.
- No existe una única respuesta correcta para todos los pasos: importa la secuencia de decisiones.

Ejemplos: elegir jugadas en ajedrez o aprender a controlar un robot caminante.

El ciclo de interacción

1. El agente observa el **estado** actual.
2. La política selecciona una **acción**.
3. El entorno responde con una **recompensa** y un nuevo estado.
4. El agente usa la experiencia para mejorar sus decisiones.

$$s_t \xrightarrow{a_t} (r_{t+1}, s_{t+1})$$

Elementos del problema

- **Agente:** toma las decisiones y aprende de la experiencia.
- **Entorno:** el mundo real o simulado con el que interactúa.
- **Estado:** información disponible para decidir en un momento dado.
- **Acción:** alternativa que el agente puede ejecutar en ese estado.

Recompensa y política

- **Recompensa:** señal inmediata que indica el resultado de una acción.
- **Política** π: regla que el agente sigue para elegir acciones según el estado.
- Las recompensas pueden ser positivas o negativas; su significado depende del objetivo.
- Una buena política busca un resultado favorable **a lo largo del tiempo**, no solo en el paso actual.

El proceso de decisión de Markov

Un MDP resume los elementos necesarios para modelar el problema:

$$\mathcal{M} = (S, A, P, R, \gamma)$$

- S: estados · A: acciones posibles.
- $P(s' \mid s, a)$: probabilidad de transición al estado s'.
- $R(s, a)$: recompensa asociada a una decisión.
- $\gamma \in [0, 1)$: cuánto peso asignamos al futuro.

De una decisión a la siguiente

- El agente parte de un estado inicial s_0 y elige una acción a_0.
- El entorno produce un nuevo estado según $P(s_1 \mid s_0, a_0)$.
- El proceso se repite: $s_0, a_0, r_1, s_1, a_1, r_2, s_2, \ldots$

La propiedad de Markov indica que el estado actual resume la información relevante para predecir el siguiente paso.

Retorno: sumar recompensas futuras

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

- Sin descuento, las recompensas lejanas pesan tanto como las inmediatas.
- Con $\gamma < 1$, se priorizan resultados próximos y la suma permanece acotada.
- El agente busca maximizar el retorno esperado.

Valor de una política

El valor $V^{\pi}(s)$ es el retorno esperado al partir del estado s y seguir la política π:

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid s_0 = s, \pi\right]$$

Permite comparar qué tan conveniente es un estado bajo una estrategia determinada.

Ecuación de Bellman

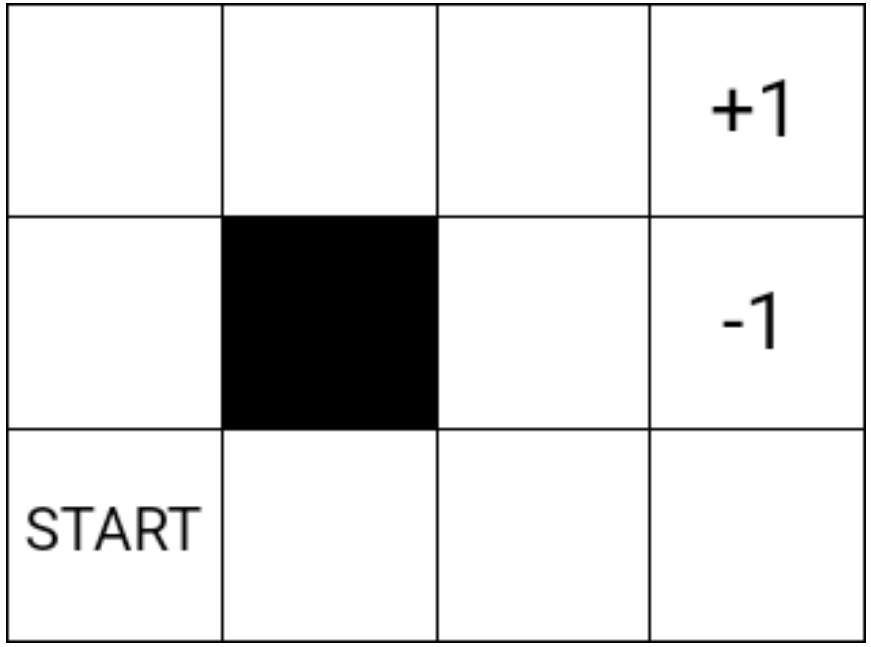
El valor de un estado se expresa en términos de su recompensa inmediata y los estados siguientes:

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^{\pi}(s') \right]$$

La ecuación relaciona la decisión actual con las consecuencias futuras.

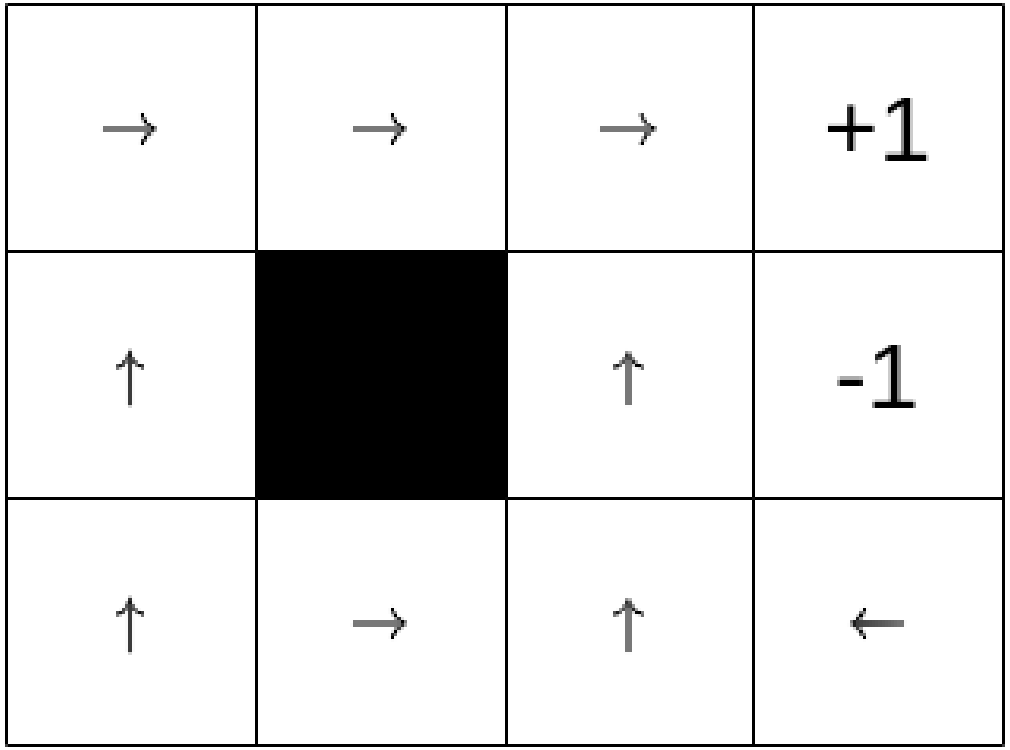
Caso de estudio: mundo grilla

- Cada casilla representa un **estado**.
- El agente puede moverse arriba, abajo, izquierda o derecha.
- Las casillas terminales entregan recompensas $+1$ y -1.



Una política asigna acciones a los estados

La política muestra qué movimiento elegir desde cada casilla para acercarse a una salida conveniente.



Mundo grilla determinista

- Cada acción produce el movimiento esperado.
- Una ruta óptima desde el inicio es $\uparrow, \uparrow, \rightarrow, \rightarrow, \rightarrow$.

V=1	V=1	V=1	+1
V=1		V=1	-1
START	V=1	V=1	V=1

El descuento puede cambiar la ruta

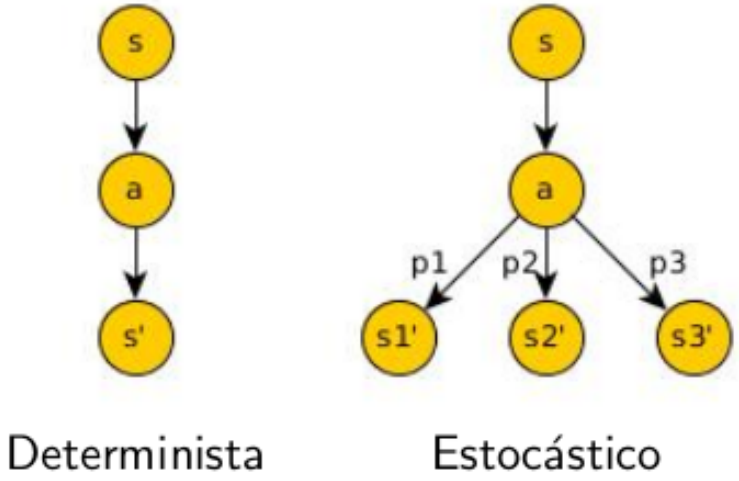
- Las recompensas futuras se multiplican por potencias de γ.
- Una ruta corta hacia una recompensa puede ser preferible a otra más larga.
- El valor de una acción depende tanto de la recompensa como de los pasos necesarios para alcanzarla.

$$\gamma = 0.9$$

V=0.81	V=0.9	V=1	+1
V=0.73		V=0.9	-1
START	V=0.73	V=0.81	V=0.73

¿Qué ocurre si las acciones son inciertas?

- La acción puede terminar en una casilla distinta de la esperada.
- Ejemplo: 80 % de probabilidad de avanzar en la dirección elegida y 10 % hacia cada lado.
- El agente debe valorar tanto la recompensa como el riesgo de cada movimiento.



Determinista Estocástico

?	?	?	+1
?		?	-1
START	?	?	?

Value Iteration: actualizar los valores

1. Inicializar $V(s)$ para todos los estados.
2. Aplicar repetidamente la actualización de Bellman:

$$V(s) \leftarrow R(s) + \gamma \max_a \sum_{s'} P(s' \mid s, a) V(s')$$

3. Detenerse cuando los valores dejan de cambiar de forma relevante.

De los valores a una política

Una vez estimado $V(s)$, elegimos la acción con mayor retorno esperado:

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V(s') \right]$$

Value Iteration permite obtener tanto los valores de los estados como una política óptima.

Q-Learning: aprender desde la experiencia

- No es necesario conocer por adelantado las probabilidades de transición.
- El agente actualiza sus estimaciones después de observar transiciones y recompensas.
- La regla incorpora la recompensa observada y el mejor valor futuro:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

¿Cómo se interpretan los valores Q?

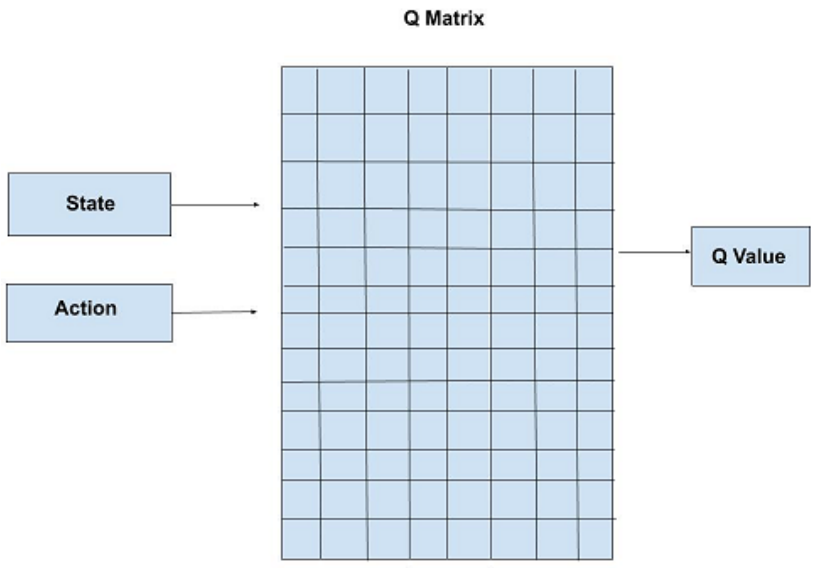
- $Q(s, a)$ estima el retorno esperado al tomar la acción a desde el estado s.
- El valor del estado es la mejor acción disponible:

$$V(s) = \max_a Q(s, a)$$

- La política codiciosa elige $\pi(s) = \arg\max_a Q(s, a)$.

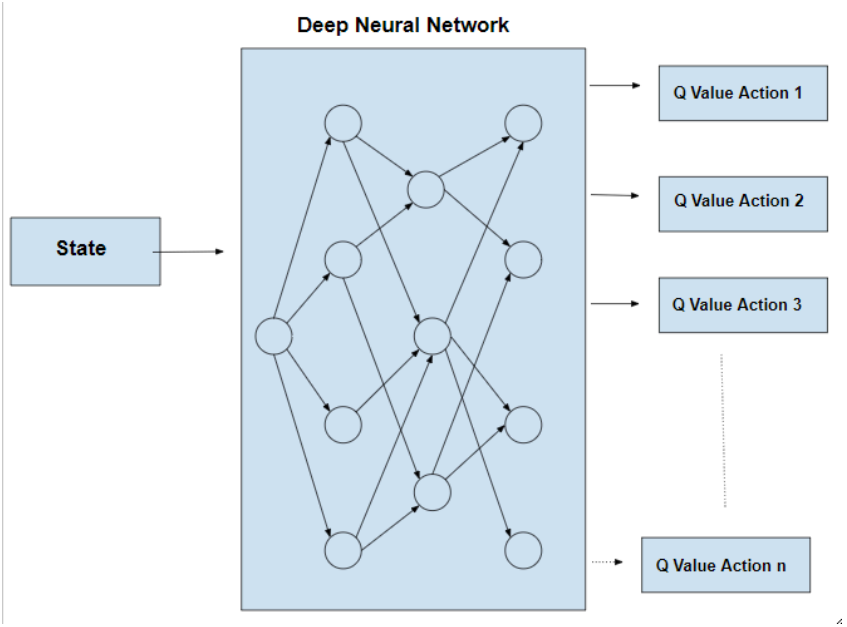
De una tabla Q a una red neuronal

Q-Learning tabular



Representa cada estado y acción en una tabla.

Deep Q-Learning



Una red estima los valores de las acciones a partir del estado.

Deep Q-Network (DQN)

- La red de política estima $Q_\theta(s, a)$ para todas las acciones posibles.
- Una red objetivo proporciona una estimación más estable del valor futuro.
- El objetivo de entrenamiento es:

$$Q_{\text{target}} = r + \gamma(1 - d) \max_{a'} Q_{\theta^-}(s', a')$$

- La pérdida reduce la diferencia entre $Q_\theta(s, a)$ y el objetivo.

Entrenamiento de un DQN

1. Elegir una acción con una política ϵ-greedy.
2. Guardar la transición (s, a, r, s', d) en una memoria de repetición.
3. Aprender con un lote aleatorio de transiciones.
4. Sincronizar periódicamente la red objetivo.

La exploración disminuye gradualmente para favorecer las acciones aprendidas.

Gymnasium: probar agentes en entornos

- Proporciona una API común para observar estados, ejecutar acciones y recibir recompensas.
- `reset()` inicia un episodio; `step(action)` devuelve estado, recompensa y señales de término.
- Incluye entornos clásicos como CartPole.

Documentación: Gymnasium.

Ejemplo práctico: DQN en CartPole

- CartPole tiene cuatro variables de estado y dos acciones posibles.
- El agente aprende a mantener el péndulo equilibrado y se evalúa con episodios nuevos.
- El notebook implementa memoria de repetición, red objetivo, entrenamiento y evaluación.

Abrir notebook PyTorch de DQN para CartPole

Ideas clave

- El aprendizaje por refuerzo optimiza decisiones secuenciales mediante retorno esperado.
- Los MDP y las ecuaciones de Bellman describen la relación entre estados, acciones y recompensas.
- Q-Learning aprende valores desde la experiencia; DQN usa redes para aproximarlos.
- La simulación permite entrenar y comparar políticas de manera reproducible.

Lecturas y referencias

- A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow*.
- Stanford, CS229 Notes.
- Documentación de Gymnasium.
- Diagrama DQN: GeeksforGeeks.